

Reachability-Guided Triage and Upstream-Verified Patching for a Real free5GC Vulnerability

[Author Name]
[University / Department]
[Country]
[email@institution.edu]

Abstract

Automated vulnerability analysis tools are frequently evaluated against synthetic reproductions of security bugs rather than the real upstream codebases those bugs were disclosed in, which limits confidence that results transfer to practice. We present a case study applying AST-based reachability filtering and rule-based patch generation to a real, disclosed vulnerability (CVE-2026-40248, CWE-285) in free5GC’s Unified Data Repository (UDR), a component of a real, standards-compliant open-source 5G core network. Reachability analysis over the real ~86KB source file reduces 102 parsed functions to the 4 that are actually reachable from the vulnerable HTTP entry points, a 96.1% reduction in analysis scope. We generate a targeted patch for each reachable finding and, going beyond isolated-file compilation, verify the patch against a live clone of the actual `free5gc/udr` repository: applying all four generated patches preserves a clean build against the project’s real dependency graph. We report this result alongside its honest limits – compile verification is not runtime confirmation, and one of the four findings is only partially corrected by the rule-based patcher – and discuss what further verification against the live network function would require.

Keywords

5G core network security, vulnerability triage, reachability analysis, program repair, free5GC

ACM Reference Format:

[Author Name]. 2026. Reachability-Guided Triage and Upstream-Verified Patching for a Real free5GC Vulnerability. In *Proceedings of 2nd free5GC World Forum, In-Cooperation with ACM SIGSAC (free5GC World Forum ’26)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Open-source 5G core network software is increasingly deployed in research testbeds, private networks, and production trials, yet security tooling aimed at such codebases is commonly validated against small synthetic examples rather than the real projects it is meant to help secure. This gap matters specifically for 5G/6G infrastructure: network functions such as free5GC’s Unified Data Repository (UDR) are large, real-world Go codebases with external dependencies (HTTP routing, database drivers, and the project’s own internal packages), and a technique that only ever compiles against an isolated copy of one file has not demonstrated it survives contact with the actual build.

This paper presents a small, fully reproducible case study that closes part of that gap for one real, disclosed free5GC vulnerability. We adopt the reachability-filtered code decomposition methodology of OpenAnt [1] – parse a codebase into functions, build a call graph, and perform breadth-first traversal from attacker-reachable entry points, which OpenAnt reports reduces analysis scope by up to 97% on real projects such as OpenSSL and WordPress – and apply it to a single, real, disclosed free5GC vulnerability. Beyond OpenAnt’s own evaluation, we add a verification step: rather than stopping at generating a patch, we apply it to a live clone of the actual upstream repository and confirm the result compiles against the project’s real dependency graph, not a scratch copy of a single file. To our knowledge this upstream-clone compile verification is not part of OpenAnt’s published evaluation or other prior reachability-based triage work.

Our contributions are: (i) an application of OpenAnt’s reachability-filtered decomposition methodology to a real free5GC source file against a real, disclosed CVE, with a 96.1% reduction in the functions requiring further analysis; (ii) automatically generated patches for all four reachable findings, verified to preserve a clean build of the real, live-cloned upstream project; and (iii) an honest accounting of what this does and does not establish, including a partially-corrected finding we report rather than omit.

2 Background

free5GC is an open-source, Apache-2.0-licensed implementation of the 5G core network, built around a Service-Based Architecture (SBA) in which each network function is an independently versioned component communicating over defined service-based interfaces. Its goal is to implement the 5GC defined in 3GPP Release 15 and beyond [2], and it has been a Linux Foundation project since September 2024, governed by a Technical Steering Committee with major contributions from National Yang Ming Chiao Tung University. The UDR (Unified Data Repository) network function exposes the `Nudr.DataRepository` service-based interface – reflected in its source layout under `internal/sbi/` – used by other network functions to read and write subscriber and policy data. Because SBA components are independently versioned, the vulnerability described below is scoped to the UDR module’s own release history, not to a free5GC project-wide version number.

CVE-2026-40248 (GHSA-jgq2-qv8v-5cmj, CVSS 3.1 base score 7.5, CWE-285 Improper Authorization) affects free5GC/udr versions $\leq 1.4.2$. The handler responsible for

creating or updating a Traffic Influence Subscription checks whether the `influenceId` path segment equals the literal string `"subs-to-notify"` and writes an HTTP 404 response when it does not – but the function does not `return` after writing that response, so execution falls through and the subscription is created or overwritten regardless of the check's outcome. An unauthenticated attacker on the 5G Service Based Interface can therefore create or overwrite arbitrary Traffic Influence Subscriptions, including attacker-controlled callback URIs, while the API misleadingly reports 404 Not Found in both the legitimate and the attack case. A sibling vulnerability in the same file and commit family, CVE-2026-40246 (GHSA-g9cw-qwhf-24jp, also CWE-285, CVSS 7.5), shows the identical missing-`return` pattern in the handler that deletes a subscription. The real fix for this vulnerability family (commit 8668627¹ on github.com/free5gc/udr) inserts the missing `return` statement across four affected handlers in `internal/sbi/api_datarepository.go`.

3 Approach

3.1 Reachability filtering

Following OpenAnt's decomposition methodology [1], we parse the target source with `tree-sitter`, a real grammar-based parser rather than a regex or brace-counting heuristic, build a call graph over all top-level function and method definitions, and perform a breadth-first traversal from a stated list of entry points – here, the four real free5GC UDR HTTP handler functions that a service-based-interface client can invoke directly. Any function not reachable from those entry points is excluded from further analysis, on the premise that code an external attacker cannot reach through the declared interface does not need to be triaged with the same priority as code that is directly exposed.

3.2 Triage and patch generation

Each reachable function is checked against a small library of rule-based vulnerability patterns. The pattern relevant to this case, `find_missing_return_after_response`, flags a function that writes an HTTP error response inside a conditional block without a following `return` statement in that block – exactly the control-flow defect described in Section 2. When the pattern matches, a patch is generated by inserting a `return` statement immediately after the flagged response write. This is a targeted, minimal-diff patch, not a full-function rewrite.

3.3 Upstream verification

Prior work verifying a generated patch typically applies it to an isolated copy of the single affected file and checks that the file alone still compiles. This does not establish that the patch is compatible with the project's actual build: a real Go module's compilation depends on its full dependency graph, declared in `go.mod/go.sum`, and on how the patched function interacts with the rest of the package. We instead clone the real

`free5gc/udr` repository at the commit immediately preceding the disclosed fix, confirm the unmodified repository builds (`go build ./...`) as a baseline, apply all generated patches directly to the real `internal/sbi/api_datarepository.go` file inside that clone, and rebuild. This exercises the project's actual dependency graph – including `gin-gonic/gin`, `go.mongodb.org/mongo-driver`, and free5GC's own internal packages – rather than a synthetic stand-in.

4 Evaluation

Reachability. Parsing `internal/sbi/api_datarepository.go` (a real, ~86KB source file) yields 102 top-level functions. Breadth-first traversal from the four real UDR entry points reaches exactly 4 of them – a 96.1% reduction in the functions a subsequent triage step must consider.

Triage. All 4 reachable functions are correctly flagged by the missing-`return` pattern and classified as CWE-285, matching the real vulnerability class disclosed for this CVE family.

Patch generation and upstream verification. A patch is generated for all 4 flagged functions. The unmodified clone builds cleanly as a baseline. With all 4 generated patches applied directly to the real, live-cloned repository, `go build ./...` again succeeds against the project's actual dependency graph. This is, to our knowledge, the first time a patch produced by this reachability pipeline has been verified against a real upstream module's full dependency graph rather than an isolated single-file copy.

A limitation surfaced by measurement, not assumption. One of the four flagged functions requires *two* separate `return` insertions in the real, disclosed fix – two independent conditional blocks in the same function body – while our rule-based patcher returns only the first match per function. That function is therefore only partially corrected: the build still succeeds (a partial fix does not break compilation), but the vulnerability is not fully closed for that one handler. We report this rather than omit it, since it is a genuine, measured limitation of a single-finding-per-pattern-match design, not a flaw discovered after publication.

5 Discussion and Limitations

A clean build after patching establishes that a generated patch is syntactically valid and compatible with the real project's dependency graph – it does not establish that the patch is correct at runtime. Confirming that the vulnerability is actually closed would require running the real free5GC UDR service and sending it a live HTTP request, which in turn requires the rest of the deployment this network function depends on: a MongoDB backend for its data store and registration with the free5GC NRF for service discovery. Standing up that deployment was outside the scope of this case study and is the most direct next step for this work. We also note this is a single-project, single-vulnerability-family case study (n=1); we do not claim the reachability reduction or patch-generation success rate observed here generalizes

¹Full hash: 86686276a7e226183ee786e3dd6714ec56c78fda

to free5GC broadly or to other 5G core implementations without further evaluation.

6 Related Work

Our reachability filtering directly follows OpenAnt [1], which combines reachability-based code decomposition with LLM-driven adversarial verification and sandboxed dynamic testing to discover previously unknown vulnerabilities across projects including OpenSSL, WordPress, and Flowise, reporting up to 97% reduction in analysis scope. We apply the same decomposition step to a single, real, disclosed CVE rather than searching for unknown bugs, and add a verification step OpenAnt’s own evaluation does not report: confirming a generated patch against a live clone of the real upstream repository, not a sandboxed exploit environment discarded after use.

DARPA’s AI Cyber Challenge (AIxCC) produced seven open-sourced cyber reasoning systems combining fuzzing, static analysis, and LLM agents for autonomous vulnerability discovery and patching [3]. OSS-CRS extends this line of work by porting AIxCC-era systems to real OSS-Fuzz projects under resource budgets [4]. Our work is narrower in scope than either – we do not attempt discovery of unknown vulnerabilities, only reachability-filtered triage and patching of an already-disclosed one – but shares their emphasis on evaluating against real, not synthetic, targets.

7 Conclusion

We presented a case study demonstrating that reachability-filtered static analysis and automated patch generation can be applied to a real, disclosed free5GC vulnerability with verification against the project’s actual upstream repository, not a synthetic reproduction – reducing 102 real functions to the 4 that matter, and confirming generated patches for all 4 preserve a clean build of the live cloned project. We report this alongside its real limits: compile-time verification is not runtime confirmation, and one finding is only partially corrected by the current rule-based patcher. We see this as a concrete, if narrow, step toward evaluating automated vulnerability tooling against real 5G core network software rather than stand-ins built for the purpose.

References

- [1] Nahum Korda and Gadi Evron. 2026. OpenAnt: LLM-Powered Vulnerability Discovery Through Code Decomposition, Adversarial Verification, and Dynamic Testing. *arXiv preprint arXiv:2606.19149* (2026). <https://arxiv.org/abs/2606.19149>
- [2] free5GC Project. <https://free5gc.org/>.
- [3] DARPA. 2025. AI Cyber Challenge marks pivotal inflection point for cyber defense. (August 2025). <https://www.darpa.mil/news/2025/aixcc-results>
- [4] Andrew Chin, Dongkwan Kim, Yu-Fu Fu, Fabian Fleischer, Youngjoon Kim, HyungSeok Han, Cen Zhang, Brian Junekeyu Lee, Hanqing Zhao, and Taesoo Kim. 2026. OSS-CRS: Liberating AIxCC Cyber Reasoning Systems for Real-World Open-Source Security. *arXiv preprint arXiv:2603.08566* (2026). <https://arxiv.org/abs/2603.08566>